

UAV-X: Resilient BVLOS Swarm Challenge

Stage 1 — Preliminary Design Verification: Technical Proposal

Team ID	TM-5A67D640440
Team Name	SwarmLink
Team ID / Registration No.	TM-5A67D640440
Institution / Organization	Indian Institute of Technology, Bombay
Team Leader	[Guna Preetham · pgunapreetham@gmail.com · +91 8712865988]
Members (up to 5)	Nachikethacharya, Denver, Guna Preetham, Yug
Category	UG + PG
Submission Date	27 September 2026
Track	GC-1 · UAV-X: Resilient BVLOS Swarm Challenge

Contents

1. Abstract / Executive Summary

Natural disasters such as earthquakes and landslides routinely destroy ground communication infrastructure, leaving a UAV swarm launched from a Ground Control Station (GCS) as often the only practical way to search an area quickly and report back in time to act. SwarmLink addresses this: 28 UAVs cover a 1000 m x 1000 m area (100 m comm range, 45-minute mission, 20-minute endurance) to locate and report 10 randomly placed, randomly timed points of interest to a fixed GCS while maintaining continuous multi-hop connectivity.

We initially targeted a ROS2 / Gazebo / ArduPilot SITL stack with a physically modelled WiFi6 link layer, aiming for hardware-representative fidelity. That stack did not come together in time, so the results below come from a lighter, fully Python discrete-event simulation with a hard-threshold connectivity model instead, section 9 gives the full account of what we attempted and why we moved on.

Our core decision is to separate task allocation from connectivity maintenance, coupling them only through pricing: a centralised CBAA-lite auction (Choi, Brunet & How)[5] with deadline-feasibility gating, so no UAV bids on a task it cannot also survive and return from, backed by a shared minimum-spanning-tree relay backbone so multiple surveyors reuse one trunk instead of each paying for a private chain, with priority- and urgency-aware slot filling and bounded preemption. Local motion uses ORCA[7] rather than an artificial potential field, avoiding APF's known local-minima deadlocks when many agents converge.

Across 10 seeds, the system achieves 100% mission and priority-weighted completion, zero collisions or geofence/battery violations, worst-case separation of 22.6 m against a 20 m requirement, and 91.4% mean network-reconfiguration efficiency.

2. Problem Understanding & Mission Analysis

Natural disaster management is a major concern all over the globe. Major Earthquakes or Landslides often disrupt terrestrial communication infrastructure, A Ground Control Station (GCS) can be established outside the affected area and deploy a swarm of UAVs.

UAVs have their own limitations in terms of bandwidth, energy, range of communication, etc. There may also be multiple points of interest, such as settlements or major road intersections, that a single robot cannot reach, hence a swarm may be deployed that can configure itself to the needs provided by the GCS.

2.1 Mission Scenario and Assumptions

Operational area	1000 m x 1000 m
------------------	-----------------

GCS	fixed, 75 m west of the area
Fleet size	28 UAVs (our choice)
Max speed	5 m/s
Comm range	100 m, hard threshold
Endurance	20 min continuous flight
Mission window	45 min, all UAVs must land by then
Max report latency	10 s from detection
Min inter-UAV separation	20 m
PoIs	10, random position, random reveal time in the first 25 min

Every value above except fleet size and the reveal window comes directly from the competition spec. **Simplifications made:** UAVs are kinematic point masses with no attitude/dynamics, the 100 m altitude spec isn't modelled (single-layer 2D sim), the comm model is a hard distance threshold, not RF propagation with signal decay.

2.2 Requirements and Objectives

- **R1 (Coverage):** Complete full sensor survey of all static and emergent disaster PoIs.
- **R2 (Connectivity):** Preserve uninterrupted multi-hop mesh connectivity — enabling BVLOS operation, to the perimeter GCS.
- **R3 (Dynamic Relays):** Automatically position non-surveying UAVs as network relay nodes.
- **R4 (Self-Healing):** Re-route links and re-auction tasks via CBBA upon vehicle failure or low battery.
- **R5 (Reprioritization):** Dynamically incorporate operator inputs and newly detected high-priority targets.
- **R6 (Safety Bounds):** Maintain a 15-meter inter-drone safety buffer, strict geofencing, and automated RTH.

2.3 Key Challenges and Trade-offs

- **Coverage vs. Connectivity:** Allocating drones to survey speeds up coverage but weakens network density.
Mitigation: CBBA utility functions weigh target proximity against network centrality, allowing agents to act as dynamic surveyor-relays along transit lines.
- **Multi-Hop Latency vs. Throughput:** High-bandwidth visual streams degrade rapidly over lossy, multi-hop channels.

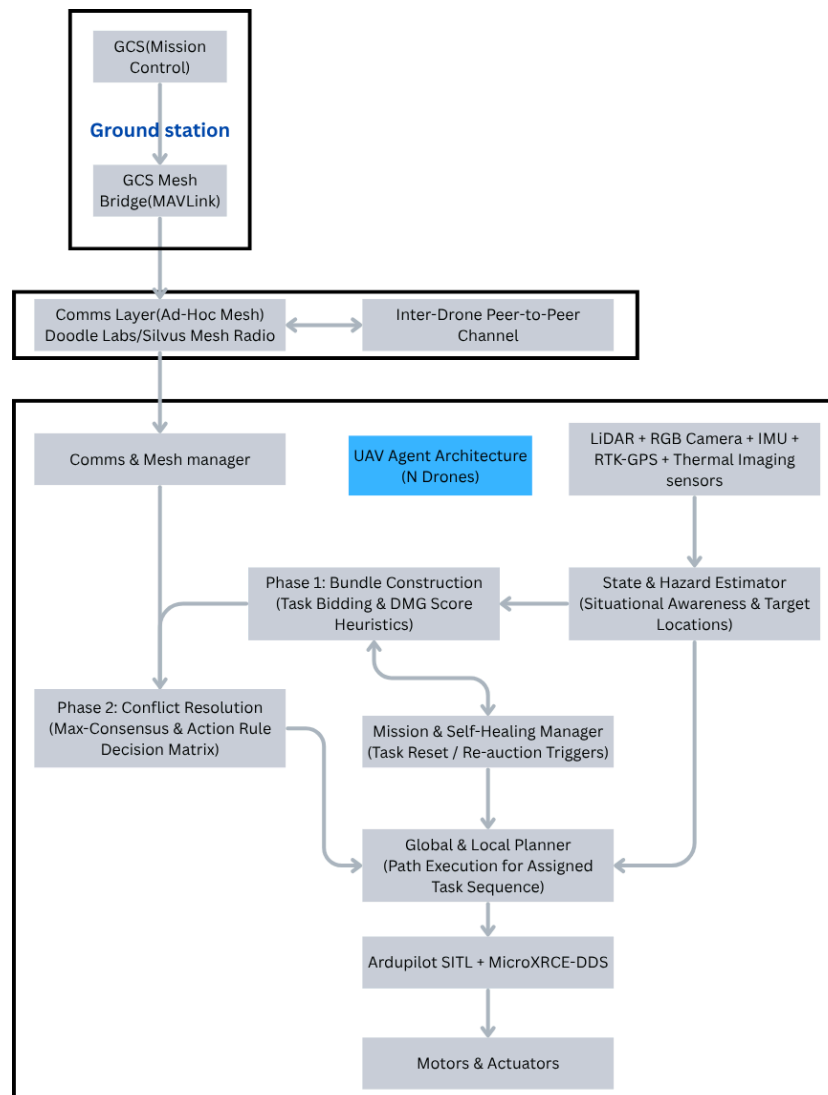
Mitigation: DDS QoS prioritizes critical state and bidding metadata, while RGB/thermal feeds use adaptive bitrate compression based on real-time link RSSI.

- **Relay Rotations vs. Topology Drops:** Withdrawing low-battery relays disconnects downstream survey drones.

Mitigation: The Mission Manager monitors State-of-Charge (<25%) to trigger preemptive CBBA relay handoffs before the departing UAV leaves position.

3. System & Software Architecture

3.1 System overview



[Figure 1: System architecture]

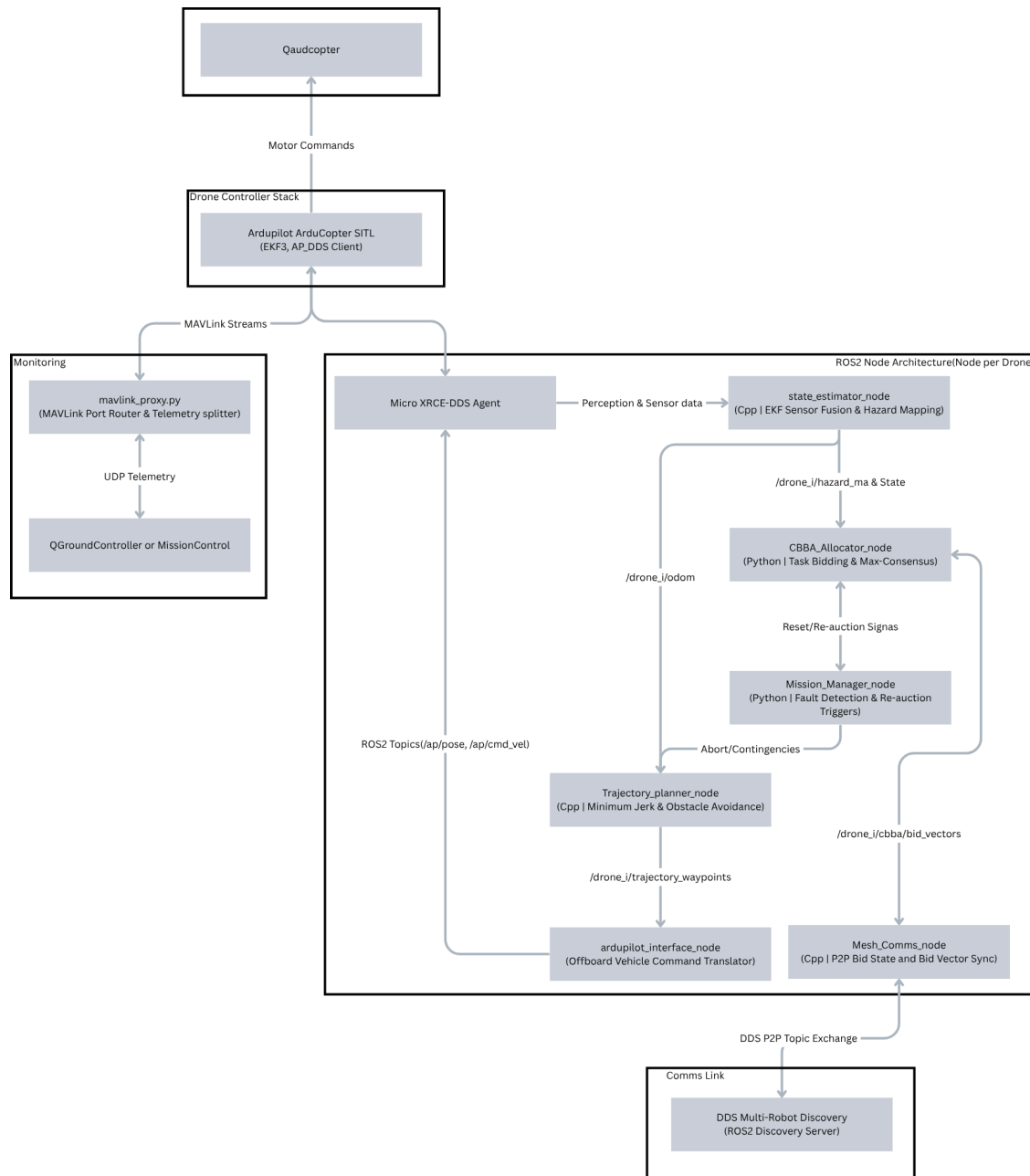
3.2 Software architecture

1. Flight Control

- **ArduPilot SITL**: Executes full autopilot firmware per UAV instance. Runs EKF3 for vehicle attitude/position estimation and outputs motor throttle commands via UDP back to Gazebo.
- **Micro XRCE-DDS Bridge**: Serves as the primary high-bandwidth, ultra-low latency command/telemetry link between ArduPilot's C++ **AP-DDS** client library and the ROS 2 environment, bypassing legacy MAVLink packet translation overhead for high-frequency control loops.
- **mavlink_proxy.py**: Intercepts MAVLink streams on dedicated port offsets ($14550 + 10 \times i$) to route telemetry to Ground Control Stations (QGroundControl) and isolate multi-instance communication.

2. High-Level ROS 2 Autonomy Stack (/drone_i)

- **State & Perception Estimator (state_estimator_node)**: Fuses sensor streams from Gazebo/Hardware with ArduPilot state telemetry to output local dynamic hazard maps, target location estimates, and obstacle occupancy grids.
- **Decentralized Task Allocation Engine (cbba_allocator_node)**: Implements Phase 1 (Bundle Construction) and Phase 2 (Max-Consensus Conflict Resolution). Computes Marginal Score Functions, maintains local bid/time vectors, and emits assigned task sequences upon convergence.
- **Trajectory & Path Planner (trajectory_planner_node)**: Receives assigned mission waypoints from CBBA. Generates kinodynamically feasible, minimum-jerk spatial trajectories while resolving collision avoidance against static obstacles and local dynamic swarm traffic.
- **Mission & Self-Healing Manager (mission_manager_node)**: Evaluates swarm system health, link dropouts, battery degradation, or target state changes. Triggers partial task resets and instructs **cbba_allocator_node** to initiate re-auctioning rounds when faults occur.
- **ArduPilot Interface Node (ardupilot_interface_node)**: Converts trajectory setpoints into native ROS 2 Micro XRCE-DDS topics (e.g., setpoint velocity, position target, or trajectory messages) expected by ArduPilot's offboard mode.
- **Swarm Comms Manager (mesh_comms_node)**: Manages inter-node P2P message exchange across the mesh network. Handles serialization, QoS tuning (transient local durability, reliable delivery), and heartbeat tracking across agents.



[Figure 2: Data-flow diagram & ROS Node Architecture]

3.3 Autonomy pipeline

Each UAV continuously runs an onboard autonomous decision loop (operating at $5\text{Hz} - 20\text{Hz}$) integrated with a **Hybrid Coordination Architecture** between the swarm and the Ground Control Station (GCS).

3.3.1 Onboard Decision Loop

1. **Perception:** Continuously collects local environmental data (optical/IR sensor streams, RSSI signal metrics, obstacle detection) and vehicle health telemetry.
2. **Situational Awareness:** Maintains active tracking of local swarm topology, neighbor trajectories, communication link quality, and newly identified Points of Interest (POIs).
3. **Task & Role Assignment:** Evaluates the onboard time-to-reach cost function $J_{i,k}$ to autonomously select between conducting a survey task (Working Swarm) or deploying as a communication relay (Networking Swarm).
4. **Planning:** Generates local trajectories via Decentralized Model Predictive Control (D-MPC), guaranteeing obstacle-free path optimization while respecting dynamic connectivity constraints.
5. **Control:** Outputs motor and flight actuator commands via the low-level flight controller (PX4/ArduPilot) for precise trajectory execution.

3.3.2 Swarm & GCS Coordination Architecture

The system utilizes a **Hybrid Coordination Strategy** designed for operational efficiency and resilience under BVLOS conditions:

- **Decentralized Swarm Execution:**
 - **Role Assignment:** Task and role allocations are dictated strictly by the onboard cost function evaluated across active UAVs, eliminating command latency over long-range wireless links.
 - **Self-Healing & Cluster Relaxation:** When a UAV becomes unresponsive, adjacent cluster nodes detect the failure within **<200 ms**. The local cluster lead recomputes "feasible" positions for remaining nodes—dynamically adjusting trajectories to cover the lost area and restore network connectivity while actively avoiding obstacles.
- **Centralized GCS Supervision:**
 - The GCS acts as a global monitoring sink, displaying swarm health, telemetry, and aggregated survey data without overriding local real-time path planning or role allocation decisions.

Architecture Justification: Fully centralized systems introduce single-point-of-failure vulnerabilities and critical latencies across BVLOS RF links. Conversely, purely decentralized systems lack central visibility. By enforcing autonomous onboard cost-function assignment and local cluster-based path recomputation, the hybrid architecture achieves rapid fault-adaptation (< 200ms) while maintaining global mission visibility at the GCS.

4. Swarm Autonomy & Mission Planning

Describe how the swarm surveys all **PoIs** efficiently within endurance limits. Cover task allocation across UAVs, coverage/path planning, revisits, and how high-priority regions that

emerge mid-mission are prioritised and re-planned.

4.1 Task allocation and coverage

Full CBBA/CBAA[5] is a distributed auction/consensus protocol. For Stage 1 we simulate the *outcome* of that process centrally rather than running message-passing agents, a stated simplification that's fair because CBAA is proven to converge to that assignment. Every 5 s, free UAVs bid on pending PoIs:

$$\text{score}(\text{uav}, \text{poi}) = \text{priority_weight}(\text{poi}) / (1 + \text{distance}(\text{uav}, \text{poi}) / D0) * \text{relay_cost_discount}(\text{poi})$$

`relay_cost_discount` is our extension: PoIs that would need a lot of new relay capacity relative to how many UAVs are free get discounted, steering the auction toward affordable PoIs before a shortage happens rather than after (a lighter version of Ponda et al. 2012's coupled task/relay bidding)[6]. Assignment is the standard sequential-greedy result CBAA/CBBA is proven to reproduce.

Deadline-feasibility gating: a UAV can't bid on a PoI unless flying there, surveying it (15 s), and flying home all fit in the time remaining. Without it, UAVs took distant assignments minutes before the deadline and never made it home, found only by letting the sim run to its actual end.

4.2 Dynamic re-tasking and priority handling

Each PoI carries one of three priority levels, weighted 1x / 2x / 4x, and that weight enters the system in three separate places rather than just the auction score above.

(i) **Task bidding**(4.1): A higher-priority PoI scores higher for any given UAV, all else equal, so the auction naturally routes free UAVs toward urgent targets first.

(ii) **Relay slot fill order**(6.2): If there aren't enough spare UAVs to fill every relay slot on the tree at once, something has to decide which slot gets a relay first. Slots are ranked by the summed urgency of the owners downstream of them. So a stranded high-priority surveyor's path back to GCS gets rebuilt before an ordinary-priority one's when the fleet can't serve both at once.

(iii) **Preemption**(6.3): That same urgency score decides whether an already-assigned relay should be pulled off a low-value edge to serve a much higher-value one.

If a UAV dies mid-task, its PoI just goes back to `PENDING` and the next auction round (every 5 seconds) picks it up like any other unassigned PoI.

4.3 Endurance and recharging management

Each UAV's battery is normalised to 1.0 and drains linearly over its 20-minute rated endurance, but only while it's actually in a flying state (transit, surveying, relaying, or returning home). Sitting idle at the GCS between assignments doesn't drain battery, a

stated Stage 1 simplification since we don't model hover power draw for the brief windows a UAV is airborne but unassigned between the 5-second replan ticks.

A UAV is forced into return-to-home the moment any of three conditions trips, whichever comes first. The simplest is battery dropping to or below a 30% threshold. The second is a time-budget check: a straight-line estimate of how long it would take to reach home, plus a 90-second safety margin. The third is an explicit flag the relay manager sets when it gives up on a cut-off owner it can no longer reach (6.4). This third trigger exists only because of a real bug we found and fixed this session: without it, a UAV cut loose by the relay manager could be left idle at a remote position with no plan to get home at all.

5. Resilient BVLOS Communication

End-to-End, multi-hop aerial link between every operating UAV and the GCS is maintained by multiple network lines

- 1) Wifi 6, 2.4GHz ([IEEE 802.11ax-2021](#)) – High speed messages
- 2) AM/FM comms – long distance communication

The network stays connected as UAVs topology changes in a mesh, as section 4 outlines the basic principles on maintaining the MST for the region of interest.

To guarantee continuous, end-to-end connectivity across the swarm and the Ground Control Station (GCS) during Beyond Visual Line-of-Sight (BVLOS) operations, our system employs an **Active Mobile Ad-Hoc Network (MANET)** with dynamic mesh routing.

Communication is categorized into two message classes:

- **Local Messages** : High-frequency, short-range peer-to-peer exchanges (e.g., Telemetry, execution heartbeats) bounded by a single physical hop.
- **Broadcast Messages** : Global state synchronization packets (e.g., target area allocations, swarm health status, emergency task preemption, etc) distributed across the entire multi-hop topology.

5.1 Communication model and assumptions

Our network stack assumes an environment where UAV nodes serve simultaneously as end-points and active routers.

- **Propagation Model:**

Using an IEEE 802.11ax path loss model – free space loss [\[0\]](#)

$$P_{rx} = P_{tx} + G_{tx} + G_{rx} - PL$$

$$PL(d) = PL(d_0) + 10 n \log_{10}(d/d_0)$$

P_{rx} : received power

P_{tx} : transmitted power

G_{tx} : transmitter gain

G_{rx} : receiver gain

PL : path loss

- **Effective Range:**

A single node is approximated to have a range of 100 m — as provided in the sample scenario — before connection becomes unstable.

The swarm, through its scalable design, however has the capacity to gather data from any far away locations – depending on the number of drones.

5.2 Multi-hop relay network design

Using the NS-3 Simulator for testing and quantifying our relay network design.

We measure connectivity on the basis of :

- 1) Latency : The time taken for a data packet to travel from the source to the destination.
- 2) Jitter : The variation in packet delay over time. It represents how much the arrival time of packets varies from one packet to another.
- 3) Packet Loss : The ratio of packets that are lost during transmission compared with the total number of packets transmitted.
- 4) Packet Error Rate: The ratio of packets that contain errors compared with the total number of received packets.
- 5) Throughput : The amount of data successfully delivered through the network per unit of time, typically measured in bits per second (bps).
- 6) Bandwidth : The amount of data that a network connection can potentially carry over a given period of time.

We use proactive OLSR (Optimized Link State Routing) for routing [\[1\]](#) due to the advantages it provides for continuously changing topologies.

This network is used to broadcast messages and maintain communication lines with the GCS.

5.3 Connectivity-aware planning

At any given moment the structure obtained using the task allocation algorithms consists of the many dynamically assigned paths that the drones will traverse.

Addressing Failure modes, on loss of a given vehicle that connects the graph of UAVs we **freeze** Paths, using the path following and relaxation methods from [\[4\]](#), reconfigure — possibly swapping roles temporarily, retrace their paths until connectivity in the graph is obtained.

6. Autonomous Relay & Role Management

6.1 Role assignment

Any idle, unassigned UAV can become a relay. The manager picks the nearest deadline-feasible free UAV for an open slot (same feasibility check as task bidding). A relay tracks its slot smoothly while the edge exists; if the edge is retired, it returns to idle.

6.2 Priority- and urgency-aware slot fill order

When multiple relay slots need filling in the same round and free UAVs are scarce, slots are ranked by how urgently they're needed, not by distance. For each edge in the relay backbone, the system checks which active surveyors depend on it (remove the edge, see who loses their path to the GCS), then scores that edge as:

$$\text{value} = \text{sum over dependent surveyors of: } \text{priority_weight} \times (1 + \text{seconds_disconnected} / 60)$$

Higher-priority surveyors and longer-waiting surveyors both push an edge's score up. Slots are filled in order of this score, highest first, with distance from the GCS only used to break exact ties. A trunk edge serving many surveyors naturally scores high because it's a sum over all of them, so shared, central parts of the backbone still tend to get filled early, but a single far-out edge carrying a stranded, high-priority surveyor can outrank it when that's genuinely more urgent.

6.3 Preemption

If a slot has had no free UAV available for 6 seconds, the system looks at relays that are already deployed elsewhere and checks whether one of them is sitting on a much less urgent edge (using the same value calculation as 6.2). "Much less urgent" means the donor edge's value must be at least 2.5× lower than the requesting slot's, a deliberately wide margin so small fluctuations in the score can't trigger a swap. The candidate also has to still be able to reach the new slot and get home afterward within the mission deadline, and it can't have been preempted in the last 15 seconds. If a valid donor is found, it's redirected to the more urgent slot.

6.4 Shortage handling and stranding recovery

If a slot stays completely unfilled for 25 seconds, the system stops waiting on it. Whichever surveyor(s) depended on that slot are released: their task goes back to **PENDING** (unless they'd already finished surveying, in which case the collected data just can't be reported), and the surveyor itself is flagged to return to the GCS immediately, rather than sitting stranded wherever it was cut off.

7. Safety & Collision Avoidance

7.1 Collision avoidance: ORCA

Collision avoidance uses ORCA (Optimal Reciprocal Collision Avoidance, van den Berg, Guy, Lin, and Manocha, 2011)[\[7\]](#). Each UAV computes its preferred velocity toward its current target, then for every other nearby UAV it constructs a half-plane of velocities that are safe with respect to that UAV, and solves for the velocity, within its max speed, that's closest to preferred while satisfying every half-plane at once. When that isn't solvable (very tight packing), it falls back to clamping onto the single most-violated half-plane rather than the full 3D solve the original paper uses, which is enough at our fleet density.

Each UAV claims half of the required 20 m separation as its own avoidance radius (10 m plus a 1.5 m buffer for timestep discretisation), so any two UAVs together claim the full 20 m. The look-ahead window is 8 seconds, tuned by sweeping 3-10 s and picking the value that gave the best observed minimum separation.

7.2 Measured safety results

Zero collisions across all 10 seeds. Worst-case minimum separation observed: 22.6 m (mean 22.8 m), measured between every pair of UAVs at every timestep, not just flagged violations. Zero geofence violations and zero battery-depletion incidents across all 10 seeds.

7.3 Geofencing and charge safety

The geofence is a rectangular boundary around the GCS and operational area, sized generously on the GCS side to accommodate the whole fleet queuing to land simultaneously near the mission deadline. Charge safety is enforced through the RTH triggers (4.3): a UAV returns home once its battery drops to 30% or its estimated time-to-return runs out, whichever comes first, so it can never fly itself into a position it can't recover from.

8. Innovation & Technical Merit

The SLAM Problem is a well studied topic, For Swarms we generally use a Pose graph and combine features matching based inter robot loop closure C-SLAM [\[2\]](#) is a framework that easily integrates with ROS2.

The Ground-Control will have more powerful computers and receive these features as well as the posegraph, enabling live reconstruction of terrain via map merging[\[3\]](#) in high resolution — this allows us to observe compromised terrain in real time and plan the path for ground vehicles, this is a considerable improvement over live satellite as the resolution of satellites are often in the range of 10m X 10m per pixel.

Repurposing 4- to 5-year-old flagship smartphones as onboard companion computers offers

a sustainable, highly cost-effective edge autonomy architecture for UAVs. Powered by multi-core ARM SoCs with integrated GPUs/NPUs, these devices combine high-density processing, high-resolution optics, and a full MEMS sensor suite (IMU, GNSS) with an internal battery that acts as an isolated UPS buffer against flight power brownouts. By flashing the smartphone with an ARM64 Linux distribution, the device natively hosts the ROS 2 autonomy stack and interfaces directly with the flight controller (PX4/ArduPilot) via a USB-to-UART serial bridge for low-latency command and telemetry exchange.

9. Simulation Framework & Proof-of-Concept

9.1 Simulation setup

(i) What we first set out to build: The PoC was originally scoped around the following stack:

- **Gazebo Harmonic** for GPU-accelerated physics and sensor rendering.
- **ArduPilot SITL** for per-vehicle autopilot flight dynamics, one dedicated SITL instance per drone model in Gazebo, synced via the [ardupilot_gazebo](#) plugin for lockstep physics.
- **ROS 2** for the high-level autonomy stack on top, with control commands and state streams carried over a **Micro XRCE-DDS** bridge, and MAVLink telemetry routed out to QGroundControl/Foxglove via a [mavlink_proxy.py](#) bridge.
- Task negotiation as an actual **distributed CBBA** implementation, agents bidding over a lossy ad-hoc mesh modeled with log-distance probabilistic path loss.
- Everything logged through ROS 2's [rosviz](#) into CSV/MCAP, so it would map directly onto the benchmark's evaluation metrics.

(ii) Why it didn't ship: Within the Stage 1 time budget, we weren't able to get this pipeline running reliably end to end across multiple simulated vehicles. Continuing to debug five layers of simulator and middleware integration would have come directly out of the time needed to design and verify the actual swarm algorithm, which is the part of this submission meant to carry the technical weight. So we set the ROS 2/Gazebo/ArduPilot stack aside for Stage 1.

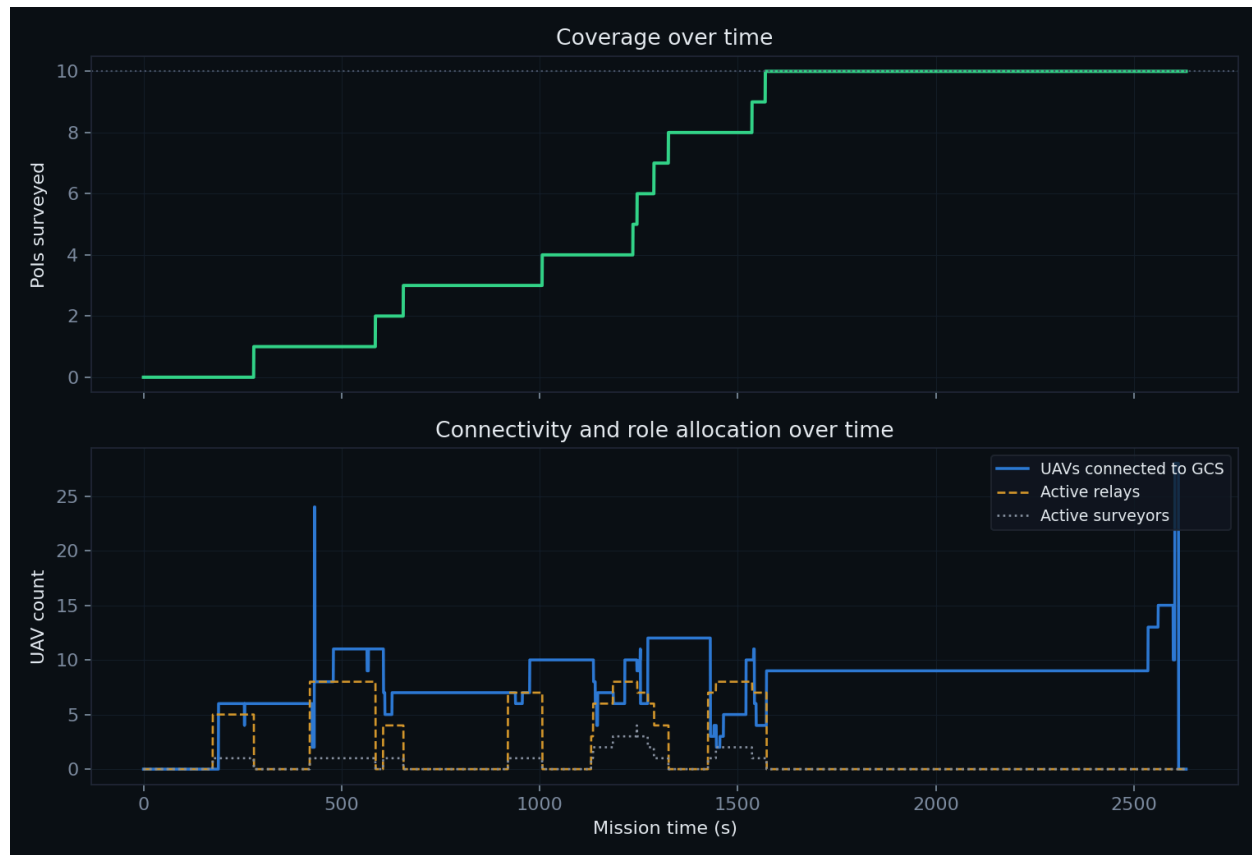
(iii) What we built instead: A from-scratch, deterministic Python simulation: kinematic point-mass UAVs, numpy for vector math, a fixed 0.5 s timestep, no external simulator or middleware dependency. Each step runs, in order:

1. CBAA auction
2. Relay update
3. Collision avoidance velocity resolution
4. Position integration
5. Battery / state-machine updates
6. PoI reporting check

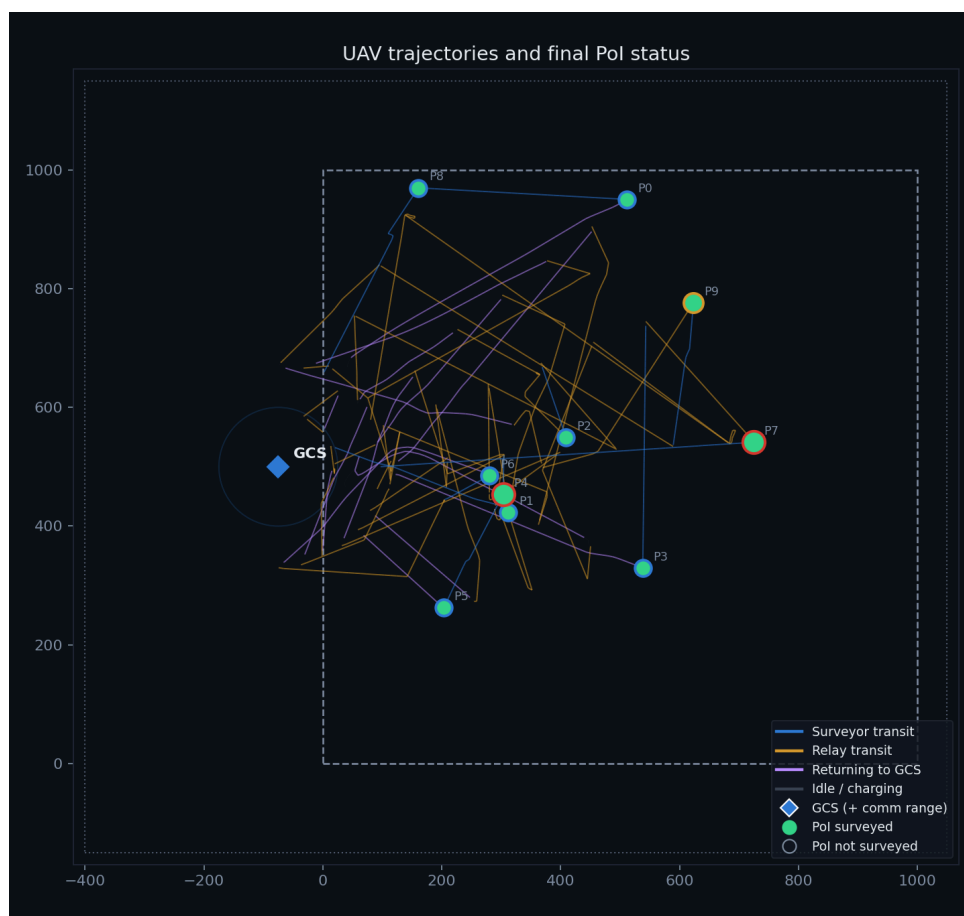
7. Metrics logging

Full source, `requirements.txt`, and a README with run instructions are in the submission.

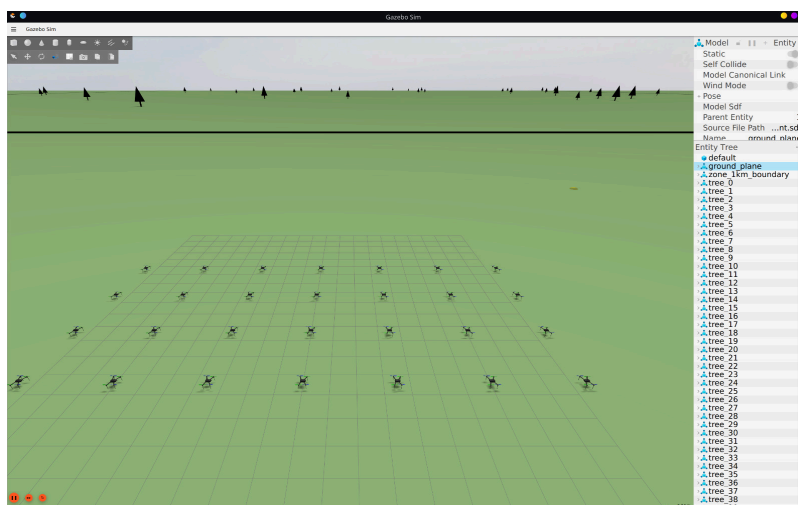
9.2 Proof-of-concept results



[Figure 3: Coverage and GCS connectivity over the 45-minute mission for seed 1, showing the relay backbone scaling with demand]



[Figure 4: UAV flight paths for seed 1, with all 10 PoIs surveyed by mission end]



[Figure 5: Gazebo world built in accordance with assumed metrics]

10. Evaluation & Performance Metrics

Report your proof-of-concept results against the official performance metrics. Use the organizers' standardised log format. State the scenario each number was measured on.

Category	Performance metric	Your result
Mission	Completion rate	[100 %]
Mission	Completion time	[2634 s mean (43.9 min)]
Mission	Priority-weighted mission score	[100% of each seed's achievable max]
Communication	Packet delivery ratio	[100 %]
Communication	Latency	1.99 ms (8.38 ms max)
Communication	Connectivity availability	[87.9 %]
Communication	Communication downtime	[612.4 s mean]
Autonomy	Relay reallocations	[98 mean]
Autonomy	Recovery time	[60s mean]
Autonomy	Network reconfiguration efficiency	[91.4% mean]
Robustness	Performance after UAV / link failures	[100% mission completed]
Safety	Collision count	[0 across all 10 seeds]
Safety	Minimum inter-UAV separation	[22.6 m worst case]

Also confirm the hard constraints: no UAV ran out of charge, and no UAV left the geofenced area.

11. Reproducibility

11.1 Installation & run instructions

Refer to the installation instructions pdf and the further README file in the container.

Repo link : <https://github.com/denverzuzarte/Nackt.git> (measurements from here)

11.2 Demonstration video

A demo html has been made and provided in the submission zip file.

12. Team & Contributions

Nachiketacharya D: responsible for the Physics Simulator, System & Software Architect

Guna Preetham Pydisetti: Responsible for swarm control and a primitive simulation, Swarm Autonomy and mission planner

Denver Zuzarte: Responsible for the Network simulator and architecture, Resilient BLVOS communication

Yug: R&D for hardware

13. References

Tools :

- 1) Gazebo: Physics simulation of environment and drone hardware
 - 2) ArduPilot: Simulated drone firmware
 - 3) NS-3: Network mesh simulation
-

Prior work :

[0] B. Bellekens, L. Tian, P. Boer, M. Weyn, and J. Famaey, "Outdoor IEEE 802.11ah Range Characterization using Validated Propagation Models," in *Proc. IEEE Global Communications Conference (GLOBECOM) Workshops*, 2017.

[0.5] A. Hazmi, J. Rinne, and M. Valkama, "Feasibility study of IEEE 802.11ah radio technology for IoT and M2M use cases," in *2012 IEEE Globecom Workshops*. IEEE, dec 2012, pp. 1687–1692. [Online]. Available: <http://ieeexplore.ieee.org/document/6477839/>

[1] Z. Nurlan, T. Zhukabayeva, and M. Othman, "Mesh Network Dynamic Routing Protocols," in *2019 IEEE 9th International Conference on System Engineering and Technology (ICSET)*, Shah Alam, Malaysia, 2019, pp. 364–369.

[2] P.-Y. Lajoie and G. Beltrame, "Swarm-SLAM: Sparse Decentralized Collaborative Simultaneous Localization and Mapping Framework for Multi-Robot Systems," *arXiv:2301.06230 [cs.RO]*, Jan. 2024.

[3] S. Yu, C. Fu, A. K. Gostar, and M. Hu, "A Review on Map-Merging Methods for Typical Map Types in Multiple-Ground-Robot SLAM Solutions," *Sensors*, vol. 20, no. 23, Art. no. 6988, 2020, doi: 10.3390/s20236988.

[4] V. S. Varadharajan, D. St-Onge, B. Adams, and G. Beltrame, "Swarm Relays: Distributed Self-Healing Ground-and-Air Connectivity Chains," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5347–5354, Oct. 2020, doi: 10.1109/LRA.2020.3006793.

[5] H. -L. Choi, L. Brunet and J. P. How, "Consensus-Based Decentralized Auctions for Robust Task Allocation," in *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912-926, Aug. 2009, doi: 10.1109/TRO.2009.2022423.

[6] S. Ponda, J. Redding, H. -L. Choi, J. P. How, M. Vavrina and J. Vian, "Decentralized planning for complex missions with dynamic communication constraints," *Proceedings of the 2010 American Control Conference*, Baltimore, MD, USA, 2010, pp. 3998-4003, doi: 10.1109/ACC.2010.5531232.

[7] van den Berg, Jur & Guy, Stephen & Lin, Ming & Manocha, Dinesh. (2011). Reciprocal n-Body Collision Avoidance. 10.1007/978-3-642-19457-3_1.

Stage 1 Submission Checklist

Confirm every item before submitting by email to pushpak_gc2026@aero.iitb.ac.in on or before 27 September 2026.

	Deliverable	Notes
☐	Technical proposal (this document)	6–8 pages, PDF
☐	Software architecture	Diagram + description (in §3 or separate)
☐	Working proof-of-concept simulation	Runs on a standard open-source framework
☐	Source code	Reproduces reported results
☐	Installation instructions	Exact, ordered steps
☐	Demonstration video	Accessible link